

SR-Peluquería

Proyecto Final - Ciclo Desarrollo de Aplicaciones Web (DAW)

Jorge Lizan Candela

Enero 2026

1. Portada y Enlaces Directos

Título del Proyecto: SR-Peluquería

Autor: Jorge Lizan Candela

Ciclo: Desarrollo de Aplicaciones Web (DAW)

Enlaces del Proyecto

Recurso	Enlace
Web Desplegada	https://peluqueria.jorgelizancandela.com
Repositorio GitHub	https://github.com/jlizancandela/proyecto
Vídeo Demo	https://youtu.be/YnsUc8a9HU8

2. Credenciales de Acceso

Para facilitar la corrección, se proporcionan las siguientes credenciales de prueba preconfiguradas en el sistema.

[Administrador]

- **Email:** test+alberto.garcia@jorgelizancandela.com
- **Contraseña:** Pelu123!

[Especialista]

- **Email:** test+maria.fernandez@jorgelizancandela.com
- **Contraseña:** Pelu123!

[Cliente de Prueba]

- **Email:** test+juan.perez@jorgelizancandela.com
- **Contraseña:** Pelu123!

Nota: No es necesario registrarse. Estos usuarios ya cuentan con datos de prueba cargados.

3. Planificación y Metodología

El desarrollo del proyecto se ha regido por **Metodologías Ágiles (Scrum)**, organizando el trabajo en **7 Sprints** bien definidos para garantizar una entrega incremental y de valor.

Tablero de Jira

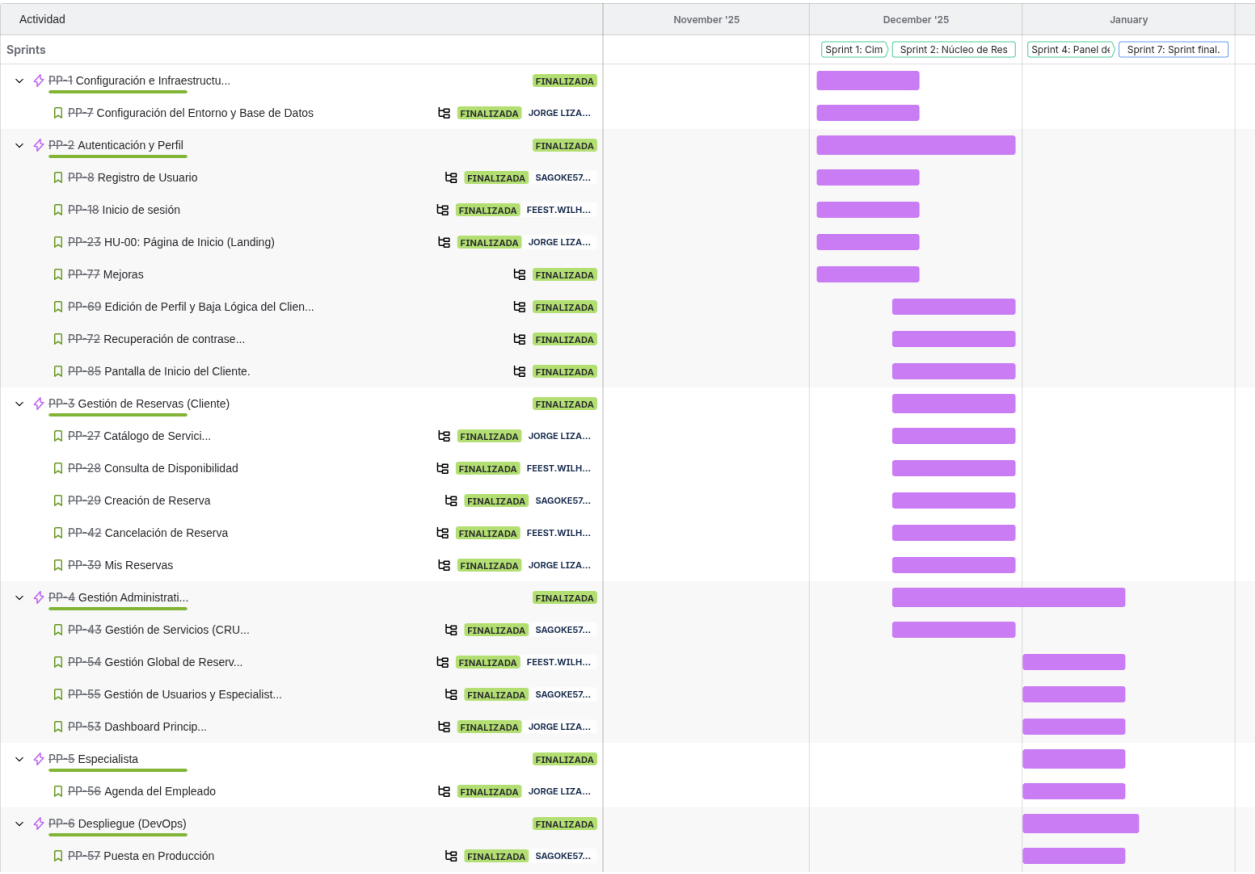


Figura 1: Cronograma de Sprints en Jira

Cierre del Proyecto: El último sprint se dedicó exclusivamente al **Despliegue y Bug Fixing**, asegurando que el entorno de producción en el VPS fuera estable, seguro y libre de errores críticos antes de la entrega final.

4. Stack Tecnológico

La arquitectura del proyecto integra tecnologías modernas de desarrollo web (DWES, DAW, DIW):

- Backend (DWES):** Desarrollado en **PHP 8.4** siguiendo un estricto **Patrón MVC** y Arquitectura Hexagonal. Se han integrado APIs externas clave:
 - Stripe:** Para la gestión segura de pagos.
 - Brevo:** Para el envío transaccional de correos electrónicos.
- Frontend (DIW):** Interfaz de usuario reactiva construida con **Preact** y **Signal** para la gestión de estado. El diseño es **Mobile First** utilizando **Bootstrap 5**, con optimización de medios a través de **ImageKit** para garantizar tiempos de carga mínimos.
- Infraestructura (DAW):** El despliegue se realiza mediante contenedores **Docker**, orquestados con Docker Compose en un **VPS de Hetzner**. La gestión del tráfico y la seguridad HTTPS se manejan mediante **Traefik** y **Cloudflare Tunnels**.

5. Sostenibilidad y Eficiencia

El proyecto se ha diseñado siguiendo criterios de sostenibilidad. Se aloja en centros de datos de Hetzner (Alemania) que utilizan energía 100% renovable. Además, se ha optimizado el tráfico de red mediante el uso de formatos de imagen WebP a través de ImageKit, reduciendo el consumo energético en la carga de la aplicación.

6. Estándares y Seguridad

Para garantizar la calidad y mantenibilidad del código, se han adoptado estándares profesionales:

- **Inglés Técnico:** Todo el código fuente (nombres de variables, clases, métodos y comentarios) está redactado en inglés, siguiendo las buenas prácticas de la industria.
 - **Seguridad:**
 - Gestión de sesiones protegidas.
 - Contraseñas almacenadas con hashing fuerte (`bcrypt`).
 - Credenciales sensibles (API Keys, contraseñas de DB) gestionadas estrictamente mediante **variables de entorno** (`.env`), sin exposición en el repositorio.
-

7. Reflexión Final

Este proyecto me ha permitido integrar un stack moderno con una gestión profesional basada en 7 Sprints en Jira, aprendiendo que la fase de QA y despliegue es tan crítica como la de código. Al implementar soluciones reales en un entorno de producción, he adquirido una visión técnica y estratégica que me prepara para afrontar retos laborales en entornos de desarrollo reales.

8. Anexo: Estructura de Carpetas

A continuación se muestra la estructura del proyecto, destacando la separación de responsabilidades y la organización del código.

```
.
|-- database/                # SQL de migraciones y datos semilla
|-- docker-compose.yml       # Orquestación de contenedores
|-- Dockerfile               # Definición de la imagen de producción
|-- public/                  # Entry point y assets estáticos
|   |-- css/
|   |-- js/
|   `-- index.php
|-- src/                      # Código Fuente (Arquitectura Modular)
|   |-- Especialistas/       # Módulo de Especialistas
|   |-- Reservas/            # Módulo de Reservas
|   |-- Servicios/           # Módulo de Servicios
|   |-- Usuarios/            # Módulo de Usuarios
|   `-- Shared/              # Utilidades compartidas
|-- tests/                   # Suite de Testeo
|   |-- Unit/                # Tests Unitarios PHP (Pest)
|   |-- js/                  # Tests Unitarios JS (Vitest)
|   `-- playwright/          # Tests E2E
`-- views/                   # Plantillas de renderizado (Latte)
    |-- components/
    |-- layouts/
    `-- pages/
```